

Adeolu Mary Oshadare

Data Science Engineer 2 - Data Pipeline & GIS Lead

GitHub: @Oshgig

Access Level: Maintainer

Reports To: @Goldokpa (Project Owner)

Project Duration: 3 Months

How This Role Fits You

Your B.Tech in Remote Sensing & GIS from FUTA gives you something no one else on this team has - a formal education in exactly the kind of spatial data ClimateVision processes. You understand satellite imagery at a fundamental level: spectral bands, atmospheric correction, spatial resolution, and coordinate reference systems.

As a GIS Analyst at Charis Tech Hub, you already worked with Google Earth Engine and AWS, writing Python scripts to model and extract insights from large geospatial datasets. That is precisely what ClimateVision's data pipeline needs - someone who can build the bridge between raw Sentinel-2 imagery and the clean, preprocessed tensors our ML models consume.

Your MSc in Data Science from Hertfordshire added the machine learning layer: Scikit-Learn, TensorFlow, XGBoost, Pandas, and data pipelines. Your credit card fraud detection project showed you can handle imbalanced datasets (SMOTE) and build production-quality ML models - the same skills needed when dealing with satellite imagery where cloud-free forest pixels are the minority class.

Your experience with Power BI, Tableau, ArcGIS Story Maps, and data storytelling means you can also create the visual outputs that make our satellite data understandable to non-technical stakeholders like conservation NGOs.

Your Role on ClimateVision

You own the entire data layer - everything that happens between raw satellite imagery arriving from APIs and clean, model-ready data being passed to the ML pipeline. You are the gatekeeper of data quality.

Core Responsibilities

- Build and maintain the automated satellite data ingestion pipeline (Sentinel Hub, Google Earth Engine)
- Develop preprocessing workflows: cloud masking, atmospheric correction, image normalization, tiling
- Create PyTorch Dataset & DataLoader classes for training and inference
- Implement data augmentation strategies (rotation, flipping, spectral perturbations)
- Engineer spectral features: NDVI, EVI, moisture indices from raw multispectral bands
- Build data validation and quality checks for incoming satellite imagery
- Manage the data/ directory structure (raw, processed, satellite)
- Create EDA notebooks for spatial data exploration and visualization

Sprint Progress - April 2026

- DONE: band_mapping.py - analysis-specific band mapping from config.yaml

- DONE: gee_downloader.py - GEE tile download with metadata
- DONE: preprocessing.py - SCL-based cloud masking
- DONE: dataset.py - PyTorch Dataset and DataLoader
- DONE: augmentation.py - training/validation transforms
- DONE: synthetic.py - synthetic fallback tile generation
- PENDING: Real GEE tile download in inference pipeline (not just NDVI stats)
- PENDING: Analysis-specific band mapping enforcement in inference preprocessing
- PENDING: Cloud masking integration at inference time before model forward pass
- PENDING: Synthetic fallback guardrails - clearly label synthetic tiles in metadata

Your Codebase Ownership

You are the primary owner of the following files and directories:

src/climatevision/data/	# PRIMARY OWNER - Entire data module
band_mapping.py	# Analysis-specific band mapping (ACTIVE)
gee_downloader.py	# GEE tile downloader (ACTIVE)
preprocessing.py	# Cloud masking, normalization (ACTIVE)
dataset.py	# PyTorch Dataset & DataLoader (ACTIVE)
augmentation.py	# Data augmentation pipeline (ACTIVE)
synthetic.py	# Synthetic tile fallback (ACTIVE)
__init__.py	# Module exports
src/climatevision/utils/	
geospatial.py	# CO-OWNER - Geospatial utilities
visualization.py	# CO-OWNER - Spatial visualizations
scripts/	
prepare_data.py	# Automated satellite data download
config.yaml	# Band mapping & analysis type config
data/	# Data directory structure
raw/ processed/ satellite/	
notebooks/	
02_data_exploration.ipynb	# EDA notebook

Your 3-Month Delivery Timeline

MONTH 1: Foundation (Weeks 1-4)

Week 1-2: Data Ingestion

- Set up Sentinel Hub API and Google Earth Engine authentication
- Build sentinel2.py - download, parse, and store Sentinel-2 imagery
- Create landsat.py - Landsat 8/9 data loader with band mapping

- Implement basic cloud masking using SCL (Scene Classification Layer)

Week 3-4: PyTorch Data Pipeline

- Build dataset.py - PyTorch Dataset class for satellite image tiles
- Implement preprocess.py - normalization, atmospheric correction, tiling (256x256)
- Create data validation checks (band count, resolution, CRS consistency)
- Write 02_data_exploration.ipynb - EDA notebook with sample visualizations

MONTH 2: Advanced Features (Weeks 5-8)

Week 5-6: Feature Engineering & Augmentation

- Implement spectral index calculation: NDVI, EVI, SAVI, moisture indices
- Build augmentation.py using albumentations (rotation, flip, spectral noise)
- Add temporal compositing - median/max NDVI composites over time windows

Week 7-8: Scale & Performance

- Integrate Dask for distributed preprocessing of large image collections
- Optimize data loading with parallel I/O and memory-mapped files
- Build data caching layer for preprocessed tiles

MONTH 3: Production Readiness (Weeks 9-12)

Week 9-10: Quality & Validation

- Implement data validation framework (schema checks, anomaly detection)
- Set up DVC (Data Version Control) for dataset tracking
- Create data quality reports and monitoring dashboards

Week 11-12: Documentation & Integration

- Write comprehensive docstrings and module documentation
- Integration testing with ML pipeline (ensure DataLoader feeds models correctly)
- Create data pipeline tutorial notebook for onboarding

Your Git Workflow

Follow this branching convention for all your work:

```
# Create feature branches from develop
git checkout develop
git pull origin develop
git checkout -b feature/data-sentinel2-loader

# Your branch naming convention:
feature/data-*          (new data features)
fix/data-*              (bug fixes in data module)
refactor/data-*         (restructuring data code)
```

All PRs go to the develop branch. PRs require at least 1 review from another team member. Tag @edoh-Onuh or

@franchise for data-related reviews since they consume your data outputs.

Your Key Collaborators

- @edoh-Onuh (ML Lead) - Your DataLoaders feed directly into their training pipeline. Coordinate on tensor shapes, normalization, and augmentation strategies.
- @franchise (Analytics Lead) - They need processed data for carbon estimation. Align on feature formats and metadata.
- Olufemi Taiwo (API Lead) - Inference pipeline uses your preprocessing code. Ensure consistency between training and inference data paths.
- @cutewizzy11 (Full-Stack) - Frontend map visualizations may need GeoJSON exports from your geospatial utils.

Your Code Pipeline

This is your end-to-end working pipeline from environment setup to pushing code.

Step 1: Environment Setup

```
# Clone and install dependencies
git clone https://github.com/Climate-Vision/ClimateVision.git
cd ClimateVision
pip install -r requirements.txt

# Authenticate Google Earth Engine
python scripts/setup_gee.py
# Follow browser prompt to authorise your GEE service account
```

Step 2: Analysis-Specific Band Mapping

config.yaml defines bands per analysis type. Your band_mapping.py reads this:

```
# config.yaml analysis type band definitions
deforestation: [B04, B03, B02, B08] # Red, Green, Blue, NIR -> 4ch -> 2cl
ice_melting:   [B02, B03, B04, B11] # Blue, Green, Red, SWIR -> 4ch -> 3cl
flooding:      [B03, B08, B11]      # Green, NIR, SWIR -> 3ch -> 3cl

# Python usage
from climatevision.data.band_mapping import get_bands_for_analysis, get_model_config

bands = get_bands_for_analysis('flooding')
# -> ['B03', 'B08', 'B11']

cfg = get_model_config('flooding')
# -> {'in_channels': 3, 'num_classes': 3, 'weights': 'models/unet_flood.pth'}
```

Step 3: Cloud Masking with SCL Band

Sentinel-2 Scene Classification Layer (SCL) pixel values:

```
# SCL band values
```

```
KEEP = [4, 5, 6, 7, 10]    # vegetation, bare soil, water, low cloud, thin cirrus
MASK = [0, 1, 2, 3, 8, 9]  # NO_DATA, SATURATED, DARK, SHADOW, med/high cloud

# Usage in preprocessing.py
from climatevision.data.preprocessing import apply_scl_cloud_mask

clean_image = apply_scl_cloud_mask(image, scl_band)
# image: (C, H, W)    scl_band: (H, W) uint8
```

Step 4: Ingest Satellite Data

```
# Download Sentinel-2 imagery for a bounding box and date range
python scripts/prepare_data.py \
  --bbox "-60,-15,-45,5" \
  --start 2023-01-01 \
  --end 2023-12-31 \
  --source sentinel2 \
  --output data/raw/amazon_2023

# Output: GeoTIFF tiles saved to data/raw/amazon_2023/
```

Step 5: Preprocess & Build Dataset

```
# Run cloud masking, normalization, and 256x256 tiling
python - <<'EOF'
from climatevision.data.preprocessing import preprocess_tiles
preprocess_tiles(
    input_dir='data/raw/amazon_2023/',
    output_dir='data/processed/amazon_2023/',
    tile_size=256,
    cloud_threshold=0.2
)
EOF

# Validate the PyTorch dataset loads correctly
python - <<'EOF'
from climatevision.data.dataset import SatelliteDataset
ds = SatelliteDataset('data/processed/amazon_2023/', split='train')
img, mask = ds[0]
print(f'Dataset size: {len(ds)} | Image shape: {img.shape} | Mask shape: {mask.shape}')
EOF
```

Step 6: Compute Spectral Indices

```
# Calculate NDVI, EVI, and moisture indices from raw bands
python - <<'EOF'
from climatevision.utils.geospatial import compute_indices
compute_indices(
    tile_dir='data/processed/amazon_2023/',
    indices=['ndvi', 'evi', 'moisture'],
    output_dir='data/processed/amazon_2023_features/'
```

```
)  
EOF
```

Step 7: Commit & Push Your Work

```
# Switch to your git identity  
source team_docs/switch_user.sh adeolu  
  
# Create a feature branch  
git checkout develop  
git pull origin develop  
git checkout -b feature/data-sentinel2-preprocessing  
  
# Stage your files  
git add src/climatevision/data/  
git add scripts/prepare_data.py  
  
# Commit  
git commit -m "feat(data): add Sentinel-2 cloud masking and tile preprocessing pipeline"  
  
# Push from your account  
git push adeolu feature/data-sentinel2-preprocessing
```